The 8085 Master Manual

Volume I: Ground Zero & The Physical Interface

A Grassroots Engineering Guide

Starting from absolute zero to architectural mastery.

"To understand the machine, we must first understand its silence."

Chapter 1: The Absolute Beginning

1.1 The Silence of the Silicon

Imagine a small, black, rectangular piece of plastic with 40 metal legs sticking out of it. This is the Intel 8085 microprocessor. If you set it on a table, it does absolutely nothing. It has no eyes to see, no ears to hear, and no inherent knowledge of the universe. It is essentially a microscopic labyrinth of silicon pathways, waiting for electricity to bring it to life.

To understand how this inert object becomes a computer, we must first unlearn how we communicate as humans. We use words, numbers, and concepts. The microprocessor understands exactly one thing: **Voltage**.

Inside the machine, there are only two states of reality:

- A wire has a high electrical pressure (usually around +5 Volts). We call this a **Logical 1**.
- A wire has no electrical pressure (0 Volts, or Ground). We call this a **Logical 0**.

Every single thing a computer does—whether it is displaying a video, calculating a trajectory, or running an operating system—is achieved by turning millions of microscopic switches (transistors) on and off, routing these +5V and 0V signals in highly specific patterns.

1.2 The Concept of Binary and Bytes

If one wire can only represent two states (0 or 1), it is not very useful for complex math. To represent larger numbers or complex instructions, engineers bundle these wires together. This bundle of wires is called a **Bus**.

In the 8085 architecture, the fundamental unit of data is built by bundling exactly 8 wires together. We call this 8-bit bundle a **Byte**.

MATHEMATICAL TRUTH OF A BYTE

If you have 8 wires, and each wire can be either a 0 or a 1, the total number of unique electrical patterns you can create is $2^8 = 256$.

Therefore, one Byte can represent any number from 0 (binary `00000000`) to 255 (binary `11111111`).

The Intel 8085 is an **8-bit microprocessor**. This specifically means that its internal brain (the Arithmetic Logic Unit, or ALU) and its data pathways are designed to swallow, process, and spit out information in chunks of 8 bits at a time. If it needs to calculate a number larger than 255, it has to do it in multiple steps, just like how a human calculates a large addition problem one column at a time.

1.3 The CPU and The Memory

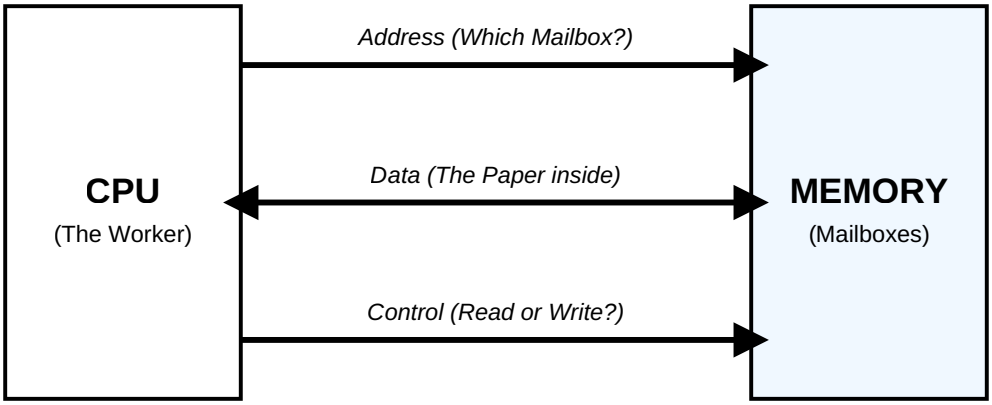
The microprocessor (CPU) is the brain, but it has severe short-term amnesia. It can calculate things incredibly fast, but it cannot store a large program inside itself. For that, it needs a partner: **The Memory**.

> **ANALOGY: THE BLIND WORKER AND THE WALL OF MAILBOXES**
>
> Imagine the microprocessor is a worker sitting at a desk in a dark room. The worker is completely blind. On the far wall of the room, there is a massive grid of mailboxes. This grid is the
> **MEMORY**
> .
>
> Each mailbox has a unique number painted on it (an *Address*). Inside each mailbox, there is a single piece of paper with an 8-bit number written on it (the *Data*).
>
> For the blind worker to do his job, he cannot walk over and look at the mailboxes. He must shout across the room: *"Send me the piece of paper currently sitting in mailbox number 2000!"* A conveyor belt then brings the paper to his desk.

This analogy perfectly describes the fundamental limitation of the microprocessor: it must have a physical way to shout the mailbox number (the Address), a physical way to receive the paper (the Data), and a physical way to tell the conveyor belt

whether to bring the paper to the desk (a Read) or take a new paper from the desk and put it in the mailbox (a Write).

These physical pathways are the **System Buses**.



Chapter 2: The Trinity of the System Bus

To connect the microprocessor to the memory and the outside world, engineers designed the **System Bus** architecture. A "bus" in computing is simply a collection of parallel copper traces on a circuit board dedicated to a specific task. The 8085 requires three distinct buses.

2.1 The Address Bus

The Address Bus is the mechanism the CPU uses to point to a specific memory location or an external peripheral device.

How wide should it be?
If the address bus only had 8 wires, the CPU could only generate $2^8 = 256$ unique addresses. A computer with only 256 bytes of memory is practically useless; it couldn't hold even a simple program.

Therefore, Intel engineers decided to give the 8085 a **16-bit Address Bus**, labelled A_0 through A_{15}. Let us examine the mathematics of this decision:

$$\text{Total Addressable Locations} = 2^{16} = 65{,}536 \text{ locations.}$$

Because each location stores exactly 1 Byte of data, the 8085 can address a maximum of 65,536 bytes. In computer terminology, 1024 bytes is 1 KiloByte (1 KB). Therefore: 65,536 / 1024 = 64. The absolute maximum physical memory capacity of the 8085 is **64 KB**.

KEY PRINCIPLE OF THE ADDRESS BUS: UNIDIRECTIONALITY

The Address Bus is strictly
UNIDIRECTIONAL
. The signals always flow *out* of the microprocessor and *into* the memory or peripherals. Memory chips do not generate addresses; they only listen for them. Therefore, the CPU is the sole master of the address bus.

2.2 The Data Bus

Once the Address Bus has successfully pointed to a mailbox, the Data Bus is used to transport the contents of that mailbox.

Because the 8085 is an 8-bit processor, its Data Bus is exactly **8 bits wide**, labelled D_0 through D_7.

KEY PRINCIPLE OF THE DATA BUS: BIDIRECTIONALITY

The Data Bus must be
BIDIRECTIONAL
.

- During a
  READ
  operation, the memory places the data on the bus, and it flows *into* the microprocessor.
- During a
  WRITE
  operation, the microprocessor places data on the bus, and it flows *out* to the memory.

This requires complex internal circuitry (tri-state buffers) to ensure the CPU and Memory do not try to talk on the bus at the exact same time, which would cause an electrical short circuit.

2.3 The Control Bus

The Control Bus is a collection of individual synchronization signals. If the Address Bus is the "Where" and the Data Bus is the "What", the Control Bus is the "When" and "How". It tells the external components exactly what the CPU intends to do. We will explore the specific signals of the Control Bus (like RD' and WR') in Chapter 5.

Chapter 3: The 40-Pin Crisis

We have established the theoretical requirements for our microprocessor. Let us attempt to build the physical chip packaging. In the late 1970s, the standard semiconductor package was the Dual In-line Package (DIP), which maxed out at **40 pins** due to manufacturing and motherboard spatial constraints.

Let us perform a hardware pin audit for our ideal 8085 chip:

Requirement	**Number of Pins Needed**
The Address Bus (A_0 - A_{15})	16 pins
The Data Bus (D_0 - D_7)	8 pins
Power Supply (+5V) & Ground (0V)	2 pins
Clock Signals (X1, X2, CLK OUT)	3 pins
Hardware Interrupts (TRAP, RSTs, INTR, INTA)	6 pins
Serial Communication (SID, SOD)	2 pins
Control & Status (RD', WR', IO/M', S0, S1, READY, HOLD, HLDA, RESET)	9 pins
Total Minimum Pins Required:	**46 pins!**

The Engineering Crisis: We need 46 pins to make the chip function, but the physical plastic package only has 40 metal legs. We are 6 pins short. How do we fit a 46-pin architecture into a 40-pin body without losing any functionality?

3.1 The Invention of Time-Division Multiplexing

Intel engineers faced a wall. They could not shrink the address bus (which would cripple memory capacity), and they could not shrink the data bus (which would cripple calculation speed).

They realized a crucial fact about the timing of the microprocessor: **The Address and the Data are never needed at the exact same microsecond.**

When the processor wants to read a memory location, the sequence of events is strictly ordered:

1. First, the CPU shouts the Address.
2. It waits a fraction of a microsecond for the memory chip to wake up and locate the data.
3. Finally, the memory chip pushes the Data onto the bus for the CPU to read.

Because step 1 and step 3 never overlap in time, the engineers decided to **share the physical pins**.

They took the lower 8 bits of the Address Bus (A_0 to A_7) and the 8 bits of the Data Bus (D_0 to D_7) and combined them onto a single set of 8 physical pins on the IC. They named these shared pins AD_0 **through** AD_7 (Address/Data).

By multiplexing these 16 theoretical signals onto 8 physical pins, they saved exactly 8 pins, bringing the total pin count down from 46 to a perfectly manufacturable 38 pins (leaving 2 pins for future design use).

Chapter 4: The Demultiplexing Dance

Multiplexing solved the physical 40-pin crisis, but it created a massive logical problem on the motherboard.

ANALOGY: THE DISAPPEARING ADDRESS

Imagine our blind worker again. To save money on conveyor belts (pins), he uses a single conveyor belt for both yelling the mailbox number (Address) and receiving the paper (Data).

At 12:00:00, he places a card on the belt that says "Mailbox 50".
At 12:00:01, he removes the card, because he needs the belt empty so the mailbox can send the paper back to him.

The problem? The memory system is slow. By the time it looks at the belt, the worker has already taken the address card away. The address has vanished into thin air!

4.1 The Need for an External Memory Pad

Because the 8085 removes the lower address from the AD_0-AD_7 pins after the first clock cycle, we must provide an external "notepad" on the motherboard to write the address down and hold it steady for the memory chips.

This external notepad is a hardware chip called a **D-Latch** (specifically, the 74LS373 Octal Transparent Latch).

4.2 The ALE Signal (Address Latch Enable)

We have a latch, but how does the latch know *when* the electrical signals on the AD_0-AD_7 pins represent an Address, and when they represent Data? If it grabs the data by mistake, it will request the wrong memory location!

To choreograph this, the 8085 utilizes a dedicated control pin: **ALE (Address Latch Enable)**.

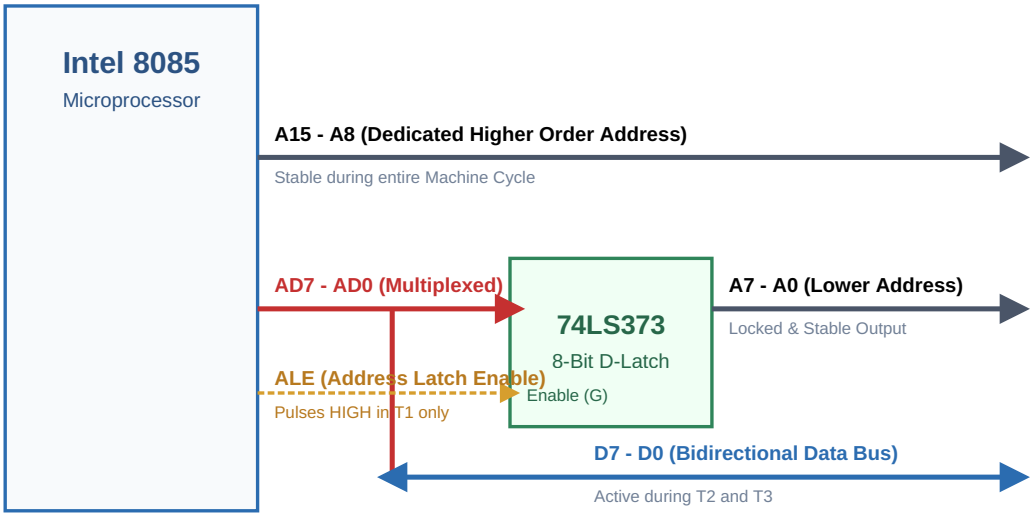
Here is the precise, microsecond-by-microsecond sequence (The Machine Cycle):

- **Clock Cycle** T_1 **(The Setup):** The 8085 begins an operation. It places the higher 8 bits of the address on the dedicated A_8-A_{15} pins. These pins are not multiplexed, so they will stay stable for the whole cycle. Simultaneously, it places the lower 8 bits of the address on the shared AD_0-AD_7 pins. Crucially, the 8085 drives the **ALE pin HIGH (+5V)**.
- **The Latch Opens:** The high ALE signal is wired directly to the 'Enable' pin of the 74LS373 external latch. When enabled, the latch becomes "transparent"—whatever electrical signals are on the AD pins flow straight through to the latch's outputs.

- **The Falling Edge of ALE:** Near the end of T_1, the 8085 drops the ALE signal back to LOW (0V). This is the trigger. As the ALE signal falls, the 74LS373 latch snaps shut, "locking" or memorizing the electrical voltages of the lower address.
- **Clock Cycles T_2 and T_3 (The Transfer):** Now that the external latch is securely holding the lower address, the 8085 abandons the address on the AD_0-AD_7 pins. It transitions those pins into a high-impedance state, waiting for the memory chip to push Data onto them. The memory chip, receiving a perfectly stable 16-bit address from the A_8-A_{15} pins and the 74LS373 latch outputs, retrieves the data and sends it back to the CPU over the now-freed AD pins.

4.3 Visualizing the Demultiplexing Architecture

To truly grasp this concept, we must visualize the physical wiring on the motherboard. Observe how the single 8-bit bus is split into two distinct buses (Address and Data) through the intervention of the ALE signal and the D-Latch.



This elegant engineering trick—multiplexing the pins and using ALE to reconstruct the buses externally—is what allowed Intel to fit the immense power of the 8085 into a standard, affordable 40-pin package, revolutionizing the computing industry.

Chapter 5: Declaring Intent (Control & Status)

We now have a stable 16-bit Address Bus pointing to a location, and a clean 8-bit Data Bus waiting for transit. But the motherboard is a crowded place. There are RAM chips, ROM chips, keyboard controllers, and display drivers all listening to the bus.

If the 8085 puts an address on the bus, how do the external devices know whether the CPU wants to *read* from that address, or *write* to that address? Furthermore, how do they know if the address belongs to a Memory chip or an Input/Output (I/O) peripheral?

The CPU resolves this ambiguity using its **Control and Status Pins**.

5.1 The IO/M' Pin (The Domain Selector)

The 8085 architecture supports two completely separate address spaces:

- A 64 KB **Memory Space** (requiring a 16-bit address).
- A 256-port **I/O Space** (requiring an 8-bit address).

The **IO/M'** (Input/Output or Memory) pin acts as a master toggle switch.

- When the CPU wants to talk to a memory chip (like a RAM module), it drives the IO/M' pin **LOW (0V)**. The bar over the 'M' denotes that it is active-low.
- When the CPU wants to talk to a peripheral (like a printer port via an `OUT` instruction), it drives the IO/M' pin **HIGH (+5V)**.

5.2 The RD' and WR' Pins (The Action Strobes)

Once the domain is selected, the CPU must declare the direction of data flow.

- **RD' (Read):** An active-low signal. When this pin goes LOW, it commands the selected memory or I/O device to place its data onto the Data Bus so the CPU can swallow it.
- **WR' (Write):** An active-low signal. When this pin goes LOW, it warns the selected memory or I/O device that the CPU has placed data onto the bus, and the device should absorb and save it.

5.3 Generating System Control Signals

Memory chips and I/O chips on the motherboard usually have specific input pins labelled MEMR' (Memory Read), MEMW' (Memory Write), IOR' (I/O Read), and IOW' (I/O Write).

The 8085 does not have these specific pins. It only provides the raw ingredients: `IO/M'`, `RD'`, and `WR'`. As systems engineers, we must combine these raw signals

using simple digital logic (typically a 3-to-8 decoder or OR gates) to generate the required system control signals.

Operation Intended	IO/M' State	RD' State	WR' State	Generated System Signal
Read data from RAM	0	0	1	**MEMR' (Memory Read)**
Write data to RAM	0	1	0	**MEMW' (Memory Write)**
Read data from Keyboard	1	0	1	**IOR' (I/O Read)**
Write data to Display	1	1	0	**IOW' (I/O Write)**

Chapter 6: Synthesis - The Complete Physical Machine

We have successfully constructed the physical foundation of the microprocessor system from absolute zero.

We began with an inert piece of silicon requiring 46 connections. Through the ingenuity of Time-Division Multiplexing (AD7-AD0), we compressed it into a 40-pin package. We solved the problem of the vanishing address by introducing the ALE signal and the 74LS373 external D-Latch, splitting the bus back into a pristine 16-bit Address Bus and an 8-bit Data Bus. Finally, we decoded the processor's intent using the IO/M', RD', and WR' signals to orchestrate the flow of data without collision.

The Transition to Phase II: The machine is now perfectly capable of reaching out to a memory chip, reading an 8-bit piece of data, and bringing it back along the Data Bus into its silicon core.

But what happens once the data enters the chip? Where does it go? How does the processor know if the data is a number to be added, or an instruction (an Opcode) telling it to jump?

To answer these questions, we must leave the physical pins behind and journey *inside* the silicon. We must explore the Internal Anatomy of the 8085—the Registers, the Arithmetic Logic Unit, and the Instruction Decoder.

That journey continues in **Phase II: The Internal Anatomy**.

End of Volume I